CS 61B Fall 2024

Final

Thursday, December 19, 2024

Your Name: _____

Your SID: _____ Location: _____

SID of person to your left: _____ Right: _____

This exam is worth **100 points**, and consists of 6 questions. Questions are ordered by topic, not by difficulty.

Question:	1	2	3	4	5	6	Total
Points:	8	15	21	18	16	22	100

- ◯ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible**.
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported, and that all code within a question belongs to the same class.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

> I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted up to 9,876,543,210 points.

_____

_____

_____

_____

_____

Signature: _____

1 Potpourri

(8 points)

(a) (1.5 points) True/False: Iterators in a Java program use the `yield` keyword instead of `return`.

◯ True ◯ False

(b) (1.5 points) True/False: If a function receives as input a list of length N, its runtime must be at least $\Omega(N)$.

◯ True ◯ False

(c) (2 points) What is the runtime of determining whether an unsorted array of N integers contains the integer 1? Select all true statements.

☐ Best-case $O(1)$

☐ Best-case $\Omega(N)$

☐ Worst-case $\Omega(\log(N))$

☐ Worst-case $\Theta(N)$

◯ None of the above

(d) (1.5 points) True/False: A Left-Leaning Red-Black Tree is always exactly twice as tall as its corresponding B-Tree.

◯ True ◯ False

(e) (1.5 points) True/False: In a Java program, calling `null.hashCode()` will result in an error.

◯ True ◯ False

2 Do It Aditya's Way!

(15 points)

Aditya is trying to come up with a way to compare superclass-subclass relationships. He decides that a superclass should be "greater than" a subclass. For example, for the following classes `Glorp`, `Foo`, and `Snapp`:

```
public class Glorp {...}
public class Foo extends Glorp {...}
public class Snapp extends Glorp {...}
```

We would say that `Glorp` $>$ `Foo`, and `Glorp` $>$ `Snapp`.
`Snapp` and `Foo` are *incomparable*; `Foo` $\not>$ `Snapp`, `Snapp` $\not>$ `Foo`, and `Foo` $\neq$ `Snapp`.

(a) (1.5 points) True/False: If class `A` $>$ `B`, and class `B` $>$ `C`, then class `A` $>$ `C`.

◯ True ◯ False

A **full ordering** of a list of classes is any ordering of classes such that, if a class `A` $>$ `B`, class `A` appears after class `B`.

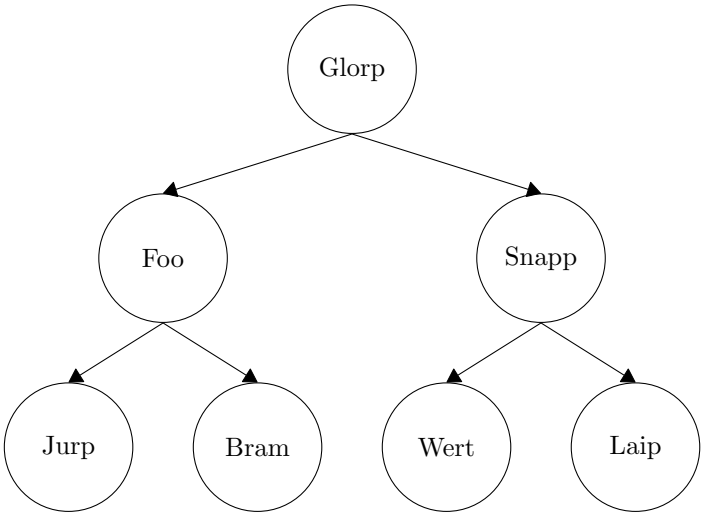
(b) (1.5 points) True/False: If class `A` appears after class `B` in a full ordering, then necessarily `A` $>$ `B`.

◯ True ◯ False

(c) (6 points) Design an algorithm for finding a full ordering given a set of classes (where each class is represented as a `String`) and a list of superclass-subclass pairs within the set. This algorithm must run in $O(N + M)$ time, where N is the number of classes and M is the total number of superclass-subclass pairs. Your algorithm may return any full ordering of classes. Describe your algorithm in no more than three sentences; no code is needed.

(d) (1 point) Given any full ordering of every non-primitive class that exists in a particular Java program, which class will always be the final element in the ordering? Answer in no more than 2 words.

Aditya now decides to place classes in a **max-heap**. All shown arrows below denote a true superclass-subclass relationship (for example, Snapp is a superclass of Wert and Laip), but **other superclass-subclass relationships may exist**.



Using the heap above, determine whether each statement will *Never Error*, *Always Error*, or *Sometimes Error* (more information is needed to determine behavior). Assume that each class contains a constructor with no arguments, and that each class has its own unique `toString` method.

(e) (1 point) `Glorp g = new Bram();`

◯ Never Error ◯ Always Error ◯ Sometimes Error

(f) (1 point) `Foo f = new Wert();`

◯ Never Error ◯ Always Error ◯ Sometimes Error

(g) (1.5 points) What is the behavior of the code below?

```
Glorp g = new Snapp();
((Wert) g).toString();
```

◯ `Glorp.toString` is always called ◯ Always Error

◯ `Wert.toString` is always called ◯ Not Enough Information

(h) (1.5 points) What is the behavior of the code below?

```
Laip l = new Wert();
((Snapp) l).toString();
```

◯ `Laip.toString` is always called ◯ Always Error

◯ `Wert.toString` is always called ◯ Not Enough Information

3 The Golden Apples of Yokeyrin

(21 points)

Anirudh the Atlas-bearing adventurer has finally reached Room 100 of Yokeyrin's cave to retrieve his real target: the Golden Apples of Yokeyrin. The goal is to give the Golden Apples to Heracles, who is currently in Room 1. They want to meet anywhere within the cave so that Anirudh can give Heracles the Golden Apples as fast as possible.

Fill out the `leastTime` method, which is defined as follows:

- Input: `Map<Integer, Set<Edge>> adj`, which maps each room (1 through 100) to a set of edges. Each `Edge` contains the room to which the `Edge` leads and the time needed to traverse the `Edge`.
- Output: The minimum time needed for Anirudh and Heracles to meet in the same room, if Anirudh and Heracles start at rooms 100 and 1, respectively.
- Heracles and Anirudh move simultaneously through the cave, and are allowed to wait in a room (without traversing an edge). Heracles and Anirudh are NOT allowed to meet in the middle of an edge.
- There is guaranteed to be at least one room where Anirudh and Heracles can meet.

```
public class State {

    public int room;
    public int time;

    public State(int room, int time) {
        this.room = room;
        this.time = time;
    }
}

public class Edge {

    public int dest;   // Destination, i.e. room where the Edge leads.
    public int weight; // Time needed to traverse the Edge.

    public Edge(int dest, int weight) {
        this.dest = dest;
        this.weight = weight;
    }
}
```

(a) (13 points) Fill out the following helper methods:

```
public class GoldenApple {

    private class StateComparator _____1_____ _____2_____ {

        @Override
        public int compare(State o1, State o2) {

            _____3_____;
        }
    }

    private Map<Integer, Integer> dijkstras(int source, Map<Integer, List<Edge>> adj) {

        Map<Integer, Integer> distTo = new HashMap<>();

        PriorityQueue<State> states = _____4_____;
        states.add(new State(source, 0));

        while (!states.isEmpty()) {

            State curr = states.poll();

            if (!distTo.containsKey(curr.room)) {

                _____5_____;

                for (Edge e : adj.get(curr.room)) {

                    State nextRoom = _____6_____;


                    _____7_____;
                }
            }
        }
        return distTo;
    }
}
```

The `State` and `Edge` classes are reprinted below for your convenience.

```
public class State {
    public int room;
    public int time;
    ...
}

public class Edge {
    public int dest;   // Destination, i.e. room where the Edge leads.
    public int weight; // Time needed to traverse the Edge.
    ...
}
```

(b) (8 points) Fill out the implementation of `leastTime` below:

```
public class GoldenApple {
    // ... continued from previous page

    public int leastTime(Map<Integer, List<Edge>> adj) {

        Map<Integer, Integer> herc = _____1_____;

        Map<Integer, Integer> ani = _____2_____;

        int lowestTime = Integer.MAX_VALUE;

        for (int i = 1; i <= 100; i++) {

            if (_____3_____ && _____4_____) {

                int currTime = _____5_____;

                lowestTime = _____6_____;
            }
        }

        return lowestTime;

    }
}
```

4 Sorting

(18 points)

Consider running in-place Heapsort on the array [7, 2, 2, 4, 5, 8, 9].

(a) (1.5 points) What is the state of the array immediately after completing bottom-up heapification?

◯ [9, 8, 5, 4, 7, 2, 2]
◯ [9, 5, 8, 4, 2, 7, 2]
◯ [2, 4, 2, 7, 5, 8, 9]
◯ [9, 5, 8, 2, 4, 2, 7]

(b) (1.5 points) What is the state of the array after exactly one iteration of Heapsort (moving one item to its correct spot and any resulting heap operations)?

◯ [5, 4, 8, 2, 2, 7, 9]
◯ [2, 5, 8, 4, 2, 7, 9]
◯ [8, 7, 4, 5, 2, 2, 9]
◯ [8, 5, 7, 4, 2, 2, 9]

(c) (1.5 points) Consider running MSD radix sort on the array [123, 312, 279, 189, 423, 275].

What is the state of the array after the first two digits have been processed?

◯ [123, 189, 275, 279, 312, 423]
◯ [123, 189, 279, 275, 312, 423]
◯ [312, 123, 423, 275, 279, 189]
◯ [312, 123, 423, 279, 275, 189]

(d) (1.5 points) Consider running LSD radix sort on the array [123, 312, 279, 189, 423, 275].

What is the state of the array after the last two digits have been processed?

◯ [123, 189, 275, 279, 312, 423]
◯ [123, 189, 279, 275, 312, 423]
◯ [312, 123, 423, 275, 279, 189]
◯ [312, 123, 423, 279, 275, 189]

For the remaining subparts, find each modified sort's best- and worst-case runtimes to sort N **distinct** items, in terms of N. If a runtime is infinite, write $\Theta(\infty)$.

Consider an extension of Mergesort, where the array is partitioned into k subarrays during each recursive step ($k = 2$ in standard Mergesort). The merging algorithm is extended to handle k subarrays:

1. During the merge step, maintain k pointers, initialized to the start of each subarray.
2. Look through all pointed elements (ignoring any pointers that are already past the end of their subarray), and select the smallest pointed element.
3. Place that item into the merged array, and advance the pointer of the corresponding subarray.
4. Repeat Steps 2 and 3 until all k pointers are past the end of their respective subarrays.

(e) (3 points) What is the runtime behavior for $k = 3$?

Best Case: $\Theta($ $)$

Worst Case: $\Theta($ $)$

(f) (3 points) What is the runtime behavior for $k = N$?

Best Case: $\Theta($ $)$

Worst Case: $\Theta($ $)$

Consider a variant of Quicksort. In this variant, the pivot is chosen as the left-most element of the array. The array is partitioned into two subarrays:

- The left subarray contains all elements strictly less than the pivot.
- The right subarray contains all elements greater than or equal to the pivot, including the pivot.

After partitioning, the pivot is placed in the right subarray. Then, the algorithm is recursively called on each subarray with more than one element.

(g) (3 points) What is the runtime behavior if the pivot is always placed at the left-most position in the right subarray?

Best Case: $\Theta($ $)$

Worst Case: $\Theta($ $)$

(h) (3 points) What is the runtime behavior if the pivot is always placed at the right-most position in the right subarray?

Best Case: $\Theta($ $)$

Worst Case: $\Theta($ $)$

5 Binary Tries

(16 points)

Consider using a trie to implement a Map, where the keys are restricted to Strings containing only `'0'`s and `'1'`s. Examples:

Valid Keys:	`"0101"`	`"0"`	`""`
Invalid Keys:	`"10015"`	`"den"`	`"123"`

Consider the `BinTrieMap` class, which implements the trie described above:

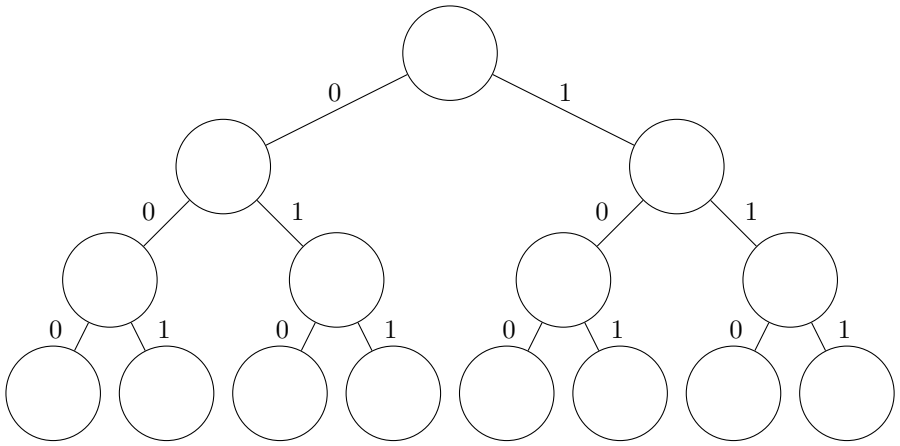
```
// Maps strings containing only '0's and '1's to non-null values.
public class BinTrieMap<V> implements Map<String, V> {
    public class Node {
        Node zero; // Child corresponding to 0.
        Node one;  // Child corresponding to 1.
        V value;   // If V is null, the string is not in the map.
                   // If V is not null, the value associated with the string.

        // Creates an empty Node, setting zero, one and value to null.
        public Node() { }
    }

    public Node root;

    @Override
    public V put(String key, V value) { /* Part (c) */ }
    ...
}
```

(a) (6 points) We create a new `BinTrieMap<Integer>`, and insert the following key-value pairs (from left to right):

Key	`"101"`	`"011"`	`"100"`	`"010"`	`"111"`	`"001"`	`"11"`	`"0"`
Value	5	6	1	2	7	4	3	8

In the below diagram of a `BinTrieMap`, write each inserted value in its corresponding node.

Not all nodes may be used.



(b) (2 points) What is the worst-case number of nodes in a `BinTrieMap` with N distinct key-value pairs, where each key is B characters long? Express your answer in Theta notation.

$\Theta($ $)$

(c) (8 points) Fill in the `put` method, which places the given key-value pair into the `BinTrieMap`, and returns the inserted value.

```
public V put(String key, V value) {

    Node n = root;

    for (char c : key.toCharArray()) { // Iterates through every character of key.

        if (_____1_____) {
            if (n.zero == null) {

                _____2_____;
            }

            _____3_____;
        }

        else if (_____4_____) {

            if (_____5_____) {

                _____6_____;
            }
            n = n.one;
        }
        else {
            throw new IllegalArgumentException("Key must contain only 0s and 1s");
        }
    }
    n.value = value;
    return value; // Returns the inserted value.
}
```

6 Natalia MST

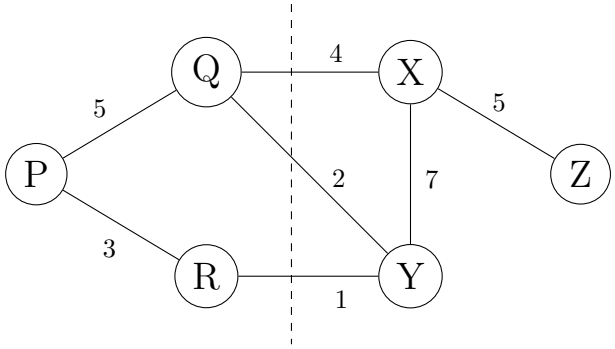
(22 points)

Reminder: Given a graph $G = (V, E)$, a cut $C = (S, T)$ of a graph is two sets of vertices S and T, where every vertex in V is in exactly one of S or T.

Given a connected graph $G = (V, E)$ with **positive** edge weights, and a cut $C = (S, T)$, the **Natalia MST** is defined as follows:

- The Natalia MST is a spanning tree of G.
- **Exactly** one edge of the Natalia MST crosses C.
- The total weight of the tree must be minimal over all spanning trees satisfying the above condition.

(a) (3.5 points) Consider the graph below, with the dotted line representing the cut C.



Select all edges in the resulting Natalia MST of the above graph.

☐ P - Q ☐ Q - X ☐ R - Y ☐ X - Z

☐ P - R ☐ Q - Y ☐ X - Y

(b) (3 points) True/False: For any connected graph G and cut $C = (S, T)$, it is always possible to generate a Natalia MST.

◯ True ◯ False

If you answered True, briefly explain your reasoning in no more than 20 words.

If you answered False, provide a counterexample by describing or drawing a graph with at most 4 nodes (draw the cut C as a dotted line).

(c) (1.5 points) True/False: For a given graph G (which has a valid Natalia MST) and a given cut C, its MST (without any restrictions) and its Natalia MST are guaranteed to have the same weight.

◯ True ◯ False

(d) (6 points) Which of the following modified MST algorithms will always produce a valid Natalia MST? Assume it is possible to construct a Natalia MST for the given graph G and cut $C = (S, T)$.
Select all that apply.

☐ Remove all edges across the cut C and run Prim's algorithm separately on the two sets S and T. Then, combine the two resulting MSTs by adding the minimum-weight edge across the cut.

☐ Identify the minimum-weight edge across the cut C, connect its two endpoints, and remove all other edges crossing the cut. Then, run Prim's algorithm starting from one of the connected nodes.

☐ Run Prim's algorithm on the entire graph. After finding the MST, keep only the minimum-weight edge across the cut C, and remove all other edges crossing the cut.

☐ Add k to all edges crossing the cut, where k is the weight of the heaviest edge in the graph. Then, run Kruskal's algorithm.

◯ None of the above

(e) (8 points) You are given a graph G, a cut C, and a corresponding valid Natalia MST, N.

An edge e is deleted from the graph, forming a new graph G'.

Design an algorithm that takes as input G, C, N, e, and G', and outputs a Natalia MST of the new graph G'.

You may assume that all graphs and Natalia MSTs are represented as adjacency lists. For credit, your algorithm must run in worst-case $O(|E|)$ runtime. You may assume that G' has a valid Natalia MST.

Note: All algorithms from the previous subpart take longer than the required runtime.

Nothing on this page is worth any points.

Bonus Questions

If you mark A, B, C, and D each with probability 1/2, what is the probability of marking at least one correct answer to this question? Select all that apply.

☐ (A) 3/4 ☐ (B) 3/4 ☐ (C) 1/4 ☐ (D) 1/8

Write the name of another class that Aditya could have put in his max-heap. As a reminder, the following class names were already used: Glorp, Foo, Snapp, Jurp, Bram, Wert, Laip.

Feedback

(0 Points)

Leave any feedback, comments, concerns, or more drawings below!